Podstawy R: operacje na wektorach atomowych, listy oraz funkcje (cz. 2)

Funkcje

Zadanie 2.1.

[MB] Napisz funkcję `calkaMonteCarlo()`. Funkcja przyjmuje cztery argumenty. Pierwszy to ściśle monotoniczna (na przedziale [`a`, `b`]), przyjmująca wartości nieujemne na przedziale [`a`, `b`] jednoargumentowa funkcja rzeczywista `f` (tego, czy jest monotoniczna, nie sprawdzaj, zakładamy, że tak jest; co do wartości nieujemnych, sprawdź je jedynie dla $f(a)$ i $f(b)$).

Kolejne argumenty, `a` i `b`, to przedział, na którym będziemy całkować funkcję `f`. Ostatnim argumentem jest liczba naturalna `n`, która będzie wyznaczać nam dokładność obliczeń (im większe `n`, tym dokładniejsze obliczenia). Niech wartość domyślna tego parametru wynosi 1000.

Funkcja ma całkować w następujący sposób:

1. niech $f_{min} = \min(f(a), f(b))$, a $f_{max} = \max(f(a), f(b))$;
2. wylosuj n punktów (x_1, x_2) z rozkładu jednostajnego na kwadracie $[a, b] \times [f_{min}, f_{max}]$ (wsk. `?runif`);
3. wyznacz frakcję punków leżących pod wykresem funkcji `f` (czyli takich że $x_2 \leqslant f(x_1)$);
4. uzyskany w punkcie 3. wynik przeskaluj mnożąc go przez miarę (powierzchnię) obszaru $[a, b] \times [f_{min}, f_{max}]$;
5. do wyniku dodaj $(b - a)f_{min}$.

Przykłady:

```
set.seed(123)
f <- function(x) sin(x)
a <- 0
b <- 1
n <- 10000
calkaMonteCarlo(f,a,b,n)

## [1] 0.461042

f <- function(x) x + 1
calkaMonteCarlo(f,a,b,n)

## [1] 1.5101

f <- function(x) x^2 + 2
calkaMonteCarlo(f,a,b,n)

## [1] 2.3304
```

Zadanie 2.2.

[MG] Dany jest wektor atomowy `x` oraz nieujemna liczba całkowita `k`. W wyniku wywołania `sample(x, k, replace=TRUE)` otrzymujemy wybrany pseudolosowo (ze zwracaniem) wektor składający się z `k` elementów z `x`. Napisz fragment kodu równoważny tej operacji. Przykład:

```
x <- c('a', 'b', 'c')
k <- 5
y <- ...
print(y)
## [1] "a" "c" "c" "a" "b"
```

Wskazówka: `runif(n, a, b)` generuje `n` obserwacji pseudolosowych z rozkładu jednostajnego na (`a`, `b`).

Zadanie 2.3.
[MG] Dany jest wektor liczb całkowitych `x`. Usuń z niego wszystkie wartości powtarzające się, nie korzystając z funkcji `unique()`, `duplicated()`, ani `anyDuplicated()`. Na przykład jeśli `x == c(1,2,1,4,3,4,1)`, to w wyniku powinniśmy otrzymać `c(1,2,4,3)`.

Zadanie 2.4.
[AC] Dane są dwa wektory `x` oraz `y` o elementach całkowitych dodatnich. Wykorzystując (koniecznie) funkcję `tabulate()`, napisz kod, który wypisze na konsolę wynik równoważny rezultatowi wywołania `intersect(x,y)`.

Zadanie 2.5.
[AC] Dany jest wektor liczbowy `x` oraz `y`. Napisz kod, który wyznaczy wartości zdefiniowanej poniżej funkcji $\widehat{F}_{\mathtt{x}}(\cdot)$ we wszystkich punktach z danego wektora `y`. Dokładniej, k-ta współrzędna wynikowego wektora jest postaci:

$$\widehat{F}_{\mathtt{x}}(y_k) = \frac{|\{x_i : x_i \leqslant y_k\}|}{|\mathtt{x}|},$$

gdzie $|\mathtt{x}|$ oznacza liczbę obserwacji w `x`. Możesz korzystać z funkcji `findInterval()` oraz `rle()` (por. zad. 2.11), lecz nie z `splinefun()` ani `approxfun()`. Zauważ, że wartości w `x` mogą się powtarzać.

Zadanie 2.6.
[MB] Napisz funkcję `posortowanePunkty()`, która jako argumenty przyjmuje: (a) wartość naturalną n, (b) wartość naturalną m oraz (c) wektor nm wartości liczbowych, przy użyciu którego dane są współrzędne m punktów w $\mathbb{R}^n$. Na przykład dla $n = 2$ i $m = 3$ podany wektor interpretujemy jako $(x_1, x_2, x_3, y_1, y_2, y_3)$, a dla $n = 3$ i $m = 2$ jako $(x_1, x_2, y_1, y_2, z_1, z_2)$.

Funkcja `posortowanePunkty()` ma za zadanie policzyć odległości (euklidesowe) punktów od początku układu współrzędnych, tj. $(0, 0, \ldots, 0)$. Następnie sortuje ona te odległości rosnąco. W wyniku jej działania otrzymujemy wektor punktów (w takim samym formacie, co wektor wejściowy), przy czym współrzędne punktów są posortowane rosnąco względem odległości od środka układu współrzędnych. (wskazówka: użyj `order()`).

Zadanie 2.7.
[MG] Napisz funkcję `approxinvert()`, która jako argumenty przyjmuje: (a) zwektoryzowaną (tj. taką, która dla n-elementowego wektora liczbowego zwraca n-elementowy wektor liczbowy) funkcję `f`, o której wiemy, że jest ciągła i ściśle monotoniczna (tego warunku nie sprawdzamy) na pewnym przedziale $[\mathtt{a}, \mathtt{b}]$; (b) wektor liczbowy `y` o elementach z $[\mathtt{f}(\mathtt{a}), \mathtt{f}(\mathtt{b})]$; (c) wartość rzeczywistą `a`; (d) wartość rzeczywistą $\mathtt{b} > \mathtt{a}$; (e) wartość całkowitą dodatnią $\mathtt{k} > 2$, domyślnie 100.

Funkcja ta ma wyznaczać przybliżenie odwrotności `f` w punktach z `y`, tj. $\mathtt{f}^{-1}(\mathtt{y})$ przy użyciu interpolacji (np. liniowej) `f` w `k` równoodległych punktach z przedziału $[\mathtt{a}, \mathtt{b}]$.

Wskazówka: możesz użyć funkcji `approxfun()` bądź `splinefun()` – zwracają one obiekty tupu funkcja.

Zadanie 2.8.
[JL] Napisz funkcję `distance()`, która dla prostej o równaniu $\mathbf{a} \cdot \mathbf{x} + b = 0$ („$\cdot$” oznacza iloczyn skalarny) w przestrzeni $\mathbb{R}^n$ wyznaczy odległość punktu $p \in \mathbb{R}^n$ od tej prostej. Odległość ta wyraża się wzorem:

$$d = \frac{|\mathbf{a} \cdot \mathbf{p} + b|}{\|\mathbf{a}\|_2},$$

gdzie $\|\mathbf{x}\|_2$ oznacza normę euklidesową wektora $\mathbf{x} \in \mathbb{R}^n$. Funkcja ta przyjmuje argumenty w domyślnej kolejności $\mathbf{p}$, $\mathbf{a}$, b.

```
distance(c(2, -2, 0.5), c(2, 1, 4), b = -4)
## [1] 0
distance(c(0, 0), c(1, 1), b = -2.0)
## [1] 1.414214
```

Zadanie 2.9 (*AC; Generowanie obserwacji z zadanego rozkładu p-stwa*)**.**
Napisz funkcję `gendyskr()`, która jako argumenty przyjmuje: (a) wartość całkowitą dodatnią `n`; (b) k-elementowy wektor liczbowy `x` o unikatowych wartościach; (c) k-elementowy wektor prawdopodobieństw `p` – jeśli wartości w `p` nie sumują się do 1, należy go przed wykonaniem obliczeń unormować i wygenerować ostrzeżenie.

Naszym zadaniem jest implementacja bodaj najprostszego algorytmu, który generuje `n`-elementową pseudolosową próbkę z rozkładu dyskretnego zmiennej losowej X określonej jako $\mathrm{P}(X = \mathtt{x}_i) = \mathtt{p}_i$ $(\forall i = 1, \ldots, k)$. Pojedynczą wartość otrzymujemy następująco:

1. Wygeneruj obserwację u z rozkładu jednostajnego na $(0, 1)$, por. `?runif`.
2. Znajdź $m \in \{1, \ldots, k\}$ takie, że $u \in (\sum_{j=1}^{m-1} \mathtt{p}_j, \sum_{j=1}^{m} \mathtt{p}_j]$, przy założeniu $\sum_{j=1}^{0} \cdot = 0$.
3. Zwróć $\mathtt{x}_m$ jako wynik.

Listy

Zadanie 2.10.
Dana jest lista

```
x <- list(a=1:10,
          b=letters[1:5],
          c=sum,
          d=seq(0, 1, 0.01),
          e=list(e1=runif(20) < 0.2,
                 e2="Ala ma kota"))
```

Znajdź:

(a) podlistę składającą się z elementów „a” oraz „b”,
(b) wartość przechowywaną jako trzeci element listy,
(c) typy wszystkich elementów listy x,
(d) podlistę, składającą się z elementów listy x, które są dłuższe niż 5,
(e) średnią każdego numerycznego elementu listy x.

Zastanów się, bez wywoływania poniszych instrukcji, co zwróci kod, jakiego typu będzie wynik i jakiej długości:

(a) `x[1]`, `x[[1]]`, `x[c(1, 2)]`, `x[[c(1, 2)]]`,
(b) `x[['c']](x[['d']])`, `x['c'](x['d'])`,
(c) `x['d'][[1]]`, `x[['d']][1]`,
(d) `x['e'][[1]][[1]]`, `x[['e']][[1]][[1]]`,
(e) `x[['e']][['e2]][1]`.

Zadanie 2.11.
[MG] Przestudiuj samodzielnie dokumentację funkcji `rle()`. Funkcja ta zwraca obiekt typu lista. Co najważniejsze, do każdego z dwóch komponentów tego obiektu (obydwa są zwykłymi wektorami atomowymi) możemy się odwołać przy użyciu wywołań:

```
rle(x)$lengths
rle(x)$values
```

Znajdź wartość, która powtarza się najczęściej (tj. modę) w danym wektorze liczb całkowitych `x`. Jeśli rozwiązanie nie jest jednoznaczne, zwróć dowolną modę.

Zadanie 2.12.
[MG] Napisz funkcję `merge_string_list()`, która złączy wszystkie napisy z danej jako argument `x` listy wektorów napisów w jeden napis. Argument `sep` (domyślnie: napis pusty) określa sposób rozdzielania poszczególnych elementów. Do testów użyj zarówno własnych list (spełniających warunki zadania, jak i takich, które powinny skutkować rzuceniem błędu) oraz danych zapisanych w plikach `zad_merge_string_list-1.json` oraz `zad_merge_string_list-2.json`. Skorzystaj w tym celu z biblioteki `jsonlite` oraz funkcji `read_json`, zwróć uwagę na parametr `simplifyVector`. Zwróć również uwagę na różnice w sposobie wypisywania napisów przy użyciu funkcji `print` oraz `cat`.

Zadanie 2.13.
[JL] Napisz funkcję `wystarczy()`, która przyjmuje jako argument listę wektorów (dowolnej długości), liczby rzeczywiste `r` oraz `R` takie, że `r` $<$ `R` oraz funkcję agregującą `fun` (dowolną funkcję przyjmującą na wejściu wektor liczb i zwracającą wartość rzeczywistą, np. `mean`). Funkcja ta zwraca listę tych wektorów z listy podanej na wejściu, dla których wartość funkcji `fun` na wektorze wynosi co najmniej `r` i co najwyżej `R`. Przykład:

```
wystarczy(list(c(6, 3, 10), c(5, 10, 15), 4), 20, 30, sum)
## [[1]]  [1] 5 10 15
```

Zalecana lektura

(1) Gągolewski, M. (2016). Programowanie w języku R. Analiza danych, obliczenia, symulacje. Wydawnictwo Naukowe PWN, Warszawa., Rozdziały: 4, 5

(2) Gagolewski M. (2026), Deep R Programming, Melbourne, DOI: 10.5281/zenodo.7490464, ISBN: 978-0-6455719-2-9, URL: https://deepr.gagolewski.com/. Rozdziały: 4, 7